TALLER PRÁCTICO: Desarmando Gigantes - Ingeniería Inversa en la Práctica

Carrera: Ingeniería de Sistemas / Software

Materia: Construcción y Evolución de Software

Capítulo 1: Ingeniería inversa, mantenimiento y evolución

Tópico: 1.1 ¿Qué es Ingeniería Inversa? Técnicas, herramientas y futuro.

Integrantes: Ruben Cuenca, Jeremy Jimenez, Alvaro Montalvan

Objetivo del Taller

Que el estudiante apropie los conceptos fundamentales de la ingeniería inversa, analice su evolución histórica y tecnológica, y aplique estos conocimientos para hipotetizar el funcionamiento de una plataforma de software moderna de alto impacto, utilizando Inteligencia Artificial como asistente de investigación y generación de contenido.

Fase 1: Calentamiento y Contextualización (Grupos de 3-4 personas)

Instrucciones:

Utilicen sus conocimientos previos y sus dispositivos para definir rápidamente los siguientes conceptos aplicados al desarrollo de software:

1. **Ingeniería Inversa:** Volver desde un producto ya hecho hacia etapas anteriores como el código.
2. **Mantenimiento de Software:**
    1. **Mantenimiento Correctivo:** Arreglar errores o fallos (bugs) del software.
    2. **Mantenimiento Adaptativo:** Modificar el software para que funcione en nuevos entornos (nuevo sistema operativo, hardware, APIs, etc.).
    3. **Mantenimiento Perfectivo:** Porque necesitas comprender el sistema actual para integrar mejoras correctamente.
    4. **Mantenimiento Preventivo:** Hacer cambios para evitar problemas futuros.
3. **Evolución del Software:** La ingeniería inversa permite analizar un sistema ya existente para entender cómo funciona, lo que facilita actualizarlo, mejorarlo o adaptarlo a nuevas tecnologías, evitando que quede inutilizable con el tiempo.

Fase 2: Investigación Asistida por IA - "Antes, Ahora y Mañana"

1. El Pasado: La Era de los Monolitos y el Bit-Hacking (Años 90)

En los 90, la ingeniería inversa se centraba en sistemas locales, a menudo con fines de interoperabilidad, corrección de errores en sistemas sin código fuente o análisis de malware incipiente.

- Entorno: Aplicaciones monolíticas ejecutándose directamente sobre el SO (MS-DOS, Windows 95/NT).
- Herramientas Clave:
    - Desensambladores (IDA Pro): El estándar de oro para convertir código máquina en lenguaje ensamblador.
    - Debuggers de Nivel de Kernel (SoftICE): Permitían detener todo el sistema operativo para inspeccionar instrucciones en tiempo real.
    - Editores Hexadecimales: Para "parchear" binarios directamente modificando bytes.
- Técnicas: Análisis estático manual y depuración paso a paso para reconstruir la lógica de algoritmos propietarios.

2. El Presente: Cloud, Microservicios y Ofuscación

Con la llegada de la web y la nube, el binario ya no es una isla; es parte de un ecosistema distribuido y altamente protegido.

- Desafíos Modernos:
    - Ofuscación Avanzada: Uso de máquinas virtuales personalizadas dentro del código (ej. VMProtect) que hacen que el desensamblado tradicional sea casi ilegible.
    - Microservicios: La lógica no está en un archivo, sino fragmentada en decenas de contenedores comunicándose vía APIs.
- Herramientas Evolucionadas:
    - Ghidra: La herramienta de la NSA que democratizó la descompilación de alta calidad (pasando de ensamblador a C casi legible).
    - Análisis Dinámico en Contenedores: Uso de herramientas de observabilidad y *eBPF* para rastrear llamadas al sistema en entornos Docker/Kubernetes.

    - Desofuscación Dinámica: Técnicas de ejecución simbólica para resolver caminos lógicos complejos automáticamente.

3. El Futuro Inmediato: Ingeniería Inversa Asistida por IA

La IA no solo acelera el proceso, sino que cambia la naturaleza de lo que es posible "entender" de un binario sin metadatos.

- Novedades y Hitos:
    - Descompilación Semántica: Los LLMs (Modelos de Lenguaje Extensos) ahora pueden tomar código ensamblador crudo y no solo traducirlo a C, sino renombrar variables y funciones con nombres lógicos basados en el contexto, recuperando la "intención" del programador original.
    - Análisis de Vulnerabilidades Automatizado: Agentes de IA pueden realizar *fuzzing* inteligente, identificando desbordamientos de búfer o fallos lógicos en segundos, tareas que antes tomaban semanas de auditoría manual.
    - Resumen de Funcionalidad: Capacidad de preguntar a una herramienta: *"¿Qué hace esta función criptográfica específica?"* y obtener una explicación en lenguaje natural.

Fase 3: El Caso de Estudio - "Hacking Hipotético"

1. Selección del Objetivo

Característica a documentar: El sistema de mensajería en tiempo real (**The Gateway**).

Hemos elegido este componente porque es el corazón de la aplicación. No se trata solo de enviar un texto, sino de la sincronización instantánea de estados (presencia), notificaciones, roles y mensajes entre millones de usuarios simultáneos.

2. Técnicas de Ingeniería Inversa

Para entender cómo funciona el Gateway sin acceso al código fuente, aplicaríamos las siguientes técnicas:

A. Análisis de Tráfico de Red (API & WebSocket Sniffing)

Esta sería la técnica principal. Dado que Discord utiliza una arquitectura basada en eventos, observaríamos el tráfico mediante:

- **Herramientas:** Wireshark o las "DevTools" del navegador (pestaña Network -> WS).

- **Objetivo:** Identificar el *handshake* del WebSocket. Analizaríamos los "Opcodes" (códigos de operación) que Discord envía para mantener la conexión viva (*Heartbeating*) y cómo los mensajes se encapsulan en formato JSON o ETF (Erlang Term Format).

B. Análisis de "Caja Negra" (Observación de Comportamiento)

Probaríamos los límites del sistema sin mirar el código:

- **Entrada/Salida:** ¿Qué sucede si enviamos 100 mensajes en un segundo? (Identificación de *Rate Limiting*).
- **Persistencia:** Si un usuario se desconecta y vuelve, ¿cómo recibe los mensajes perdidos? Esto nos indica si hay un buffer temporal o una consulta inmediata a la base de datos.

C. Ingeniería Inversa del Cliente (Descompilación)

- **Móvil (APK):** Usaríamos herramientas como jadx para descompilar el archivo de Android y buscar constantes de configuración, URLs de los servidores de Gateway y la lógica de reintento de conexión.
- **Escritorio:** Al ser una App basada en Electron, inspeccionaríamos los archivos .asar para leer el código JavaScript que maneja la lógica del lado del cliente.

3. Hipótesis de la Arquitectura

Basándonos en la alta concurrencia de Discord y su conocido uso de lenguajes como **Elixir** y **Rust**, esta es la arquitectura lógica hipotética:

Componentes Principales:

1. **Cliente (Frontend):** (Electron/React Native) Mantiene una conexión persistente vía WebSocket.
2. **Gateway (Microservicio de Tiempo Real):** Un clúster de servidores (probablemente en Elixir por su manejo de procesos livianos) que gestiona las conexiones activas.
3. **API de Mensajería (Backend):** Recibe las peticiones HTTP POST cuando un usuario envía un mensaje, lo valida y lo guarda.
4. **Base de Datos (ScyllaDB/Cassandra):** Elegida por su capacidad de escritura masiva y baja latencia para almacenar el historial de chats.

5. **Capa de Pub/Sub (NATS/Redis):** Un sistema de mensajería interna que avisa al Gateway: "Oye, hay un nuevo mensaje para el Canal X, envíalo a todos los usuarios conectados en ese canal".

Flujo Lógico:

- **Paso 1:** El Usuario A envía un mensaje (HTTP POST a la API).
- **Paso 2:** La API valida permisos y lo escribe en la **Base de Datos**.
- **Paso 3:** La API publica un evento en el **Pub/Sub**.
- **Paso 4:** El **Gateway** detecta el evento y lo empuja (*push*) a través de los WebSockets abiertos de todos los usuarios que están viendo ese canal (Usuario B, C, etc.).
- **Paso 5:** El cliente recibe el JSON y actualiza la interfaz sin necesidad de refrescar.

Fase 4: Generación de Imagen para la Exposición (Uso de IA)

Un render 3D isométrico estilo cyberpunk de una enorme estructura con el logo de Discord flotando en el centro, representando la aplicación como una máquina compleja. Un equipo de ingenieros futuristas está realizando ingeniería inversa, desmontando cuidadosamente partes externas de la estructura con herramientas digitales holográficas. En el interior se revelan componentes brillantes y detallados etiquetados como "Protocolo WebSocket", "Servidor de Voz", "Sistema de Roles" y "API REST". Flujos de datos en color azul y púrpura circulan entre los componentes, mostrando la comunicación en tiempo real. El ambiente tiene iluminación neón, alta tecnología, fondo oscuro, estilo ultra detallado, alta resolución, enfoque cinematográfico.

Fase 5: Exposición (5 minutos por grupo)

Estructura de la Exposición:

1. **Punto de Partida:** Breve mención de lo más relevante que aprendieron sobre la historia/herramientas de ingeniería inversa (Fase 2).
2. **El Reto:** ¿Qué App y qué funcionalidad eligieron analizar? ¿Por qué es un reto de ingeniería?
3. **La Estrategia:** ¿Qué técnicas modernas de ingeniería inversa (vistas en clase o investigadas con IA) aplicarían para deslegajar ese sistema?
4. **La Revelación (La Imagen):** Muestren su imagen generada por IA y expliquen cómo representa visualmente la arquitectura hipotética que descubrieron.